Comparative and Adaptive Evaluation of Unsupervised Machine Learning Models for Multivariate Sensor Anomaly Detection in Industrial Systems

A Cross-Dataset and Edge-Deployability Study

Sarjan Shah and Abhi Shah

Department of Information Technology, Lok Jagruti University, Vadodara, Gujarat, India
shahsarjan968@gmail.com, abhi.shah@ljku.edu.in

Abstract. Industrial machinery increasingly carries many sensors whose readings can reveal early faults, but labelled fault data is rarely available. This motivates unsupervised anomaly detection, which learns only from normal operation. This paper compares six unsupervised detectors (a statistical z-score baseline, One-Class SVM, Local Outlier Factor, Principal Component Analysis (PCA) reconstruction error, the Elliptic Envelope, and Isolation Forest) on two real multivariate sensor benchmarks: the Skoltech Anomaly Benchmark (SKAB) and the Secure Water Treatment (SWaT) testbed. The same leakage-free feature set and a strictly chronological, validation-tuned protocol are used throughout. Alongside F1, results are reported with the Matthews correlation coefficient (MCC) and balanced accuracy, because F1 can reward a detector that simply flags almost every reading. Three findings emerge. First, the best classical detector depends on the dataset. Isolation Forest leads on SKAB, while the correlation-based Elliptic Envelope leads on SWaT (MCC 0.77), so a single winning model does not transfer. A lightweight LSTM-autoencoder deep baseline, added for comparison, sharpens this: it is the best detector on SWaT (MCC 0.84) but falls below chance on the smaller SKAB, so deep models do not remove the dataset dependence. Second, a deployed static detector degrades under concept drift. On SWaT, where normal operation drifts over time, a static model's false-alarm rate on normal data rises from 8% toward 100%. A lightweight, CPU-only drift detector that retrains on recent data restores skill (MCC 0.24 to 0.59 on SWaT), roughly halves false alarms, and keeps 90% of attack recall. Third, the full pipeline (the classical models and the drift adaptation, on both datasets) runs inside a container limited to Raspberry-Pi-class resources of one CPU core, 1 GB RAM, and the ARM64 architecture. Inference stays below a millisecond per reading and peak memory stays under 500 MB. The study contributes a transparent multi-dataset baseline, evidence that drift adaptation is both needed and feasible on commodity hardware, and a template for careful unsupervised evaluation on real sensor data.

Keywords: anomaly detection · unsupervised learning · multivariate time series · concept drift · edge computing · predictive maintenance

1 Introduction

Modern industrial systems such as pumps, motors, valves, and their connecting pipework are routinely instrumented with sensors that measure vibration, pressure, temperature, current, and flow. These signals carry early evidence of developing faults. Detecting that evidence before failure is the goal of condition monitoring and predictive maintenance, where machine-learning methods have become central [1]. The main practical difficulty is that labelled fault data is scarce. Faults are rare, varied, and expensive to induce or annotate, so a supervised classifier trained on known faults tends to generalise poorly to the next, unseen failure mode [2].

Unsupervised anomaly detection avoids this problem. Instead of separating "normal" from labelled "fault" examples, an unsupervised (or novelty) detector is trained only on data that represents healthy operation. At inference time it flags readings that depart far enough from the learned notion of normal. This setting matches industrial practice, where normal data is plentiful but fault labels are not [3]. Many such detectors exist, ranging from simple statistical thresholds to deep reconstruction networks [4]. For a practitioner the useful question is not which method is most sophisticated, but which method actually performs best on real sensor data under a fair evaluation, and whether it keeps working once deployed.

That question is often answered on a single dataset, with optimistic evaluation protocols, or without controlling for information leakage, all of which inflate reported scores. It is also usually answered under the implicit assumption that the data distribution stays fixed. Two consequences follow, and this paper addresses both. A model declared best on one benchmark need not be best on another. And a model that is deployed and then left unchanged can fail quietly as the monitored system drifts away from its training distribution.

This paper makes the following contributions.

- It compares **six** classical unsupervised detectors, and a lightweight LSTM-autoencoder deep baseline, on **two** real industrial benchmarks (SKAB and SWaT) using one shared feature representation, so that the comparison reflects the detector and not the preprocessing.
- It follows a conservative protocol: chronological, per-experiment splits with no shuffling; an unsupervised training set free of labelled anomalies; and validation-based hyperparameter selection checked for "flag-everything" degeneracy. MCC and balanced accuracy are reported alongside F1 because they do not reward that degeneracy, and a wall-clock feature leak found during the study is documented and removed.
- It shows that the **best detector depends on the dataset**, and that this holds for deep models as well. Among the classical detectors, Isolation Forest

leads on SKAB and the correlation-based Elliptic Envelope leads on SWaT. The deep baseline is the best detector of all on SWaT but falls below chance on the smaller SKAB. A mechanistic explanation is given in terms of sensor count, correlation, and data volume.
- It reports a **concept-drift adaptation** result. A static detector falls to chance-level skill under a regime shift on SKAB, and toward flagging all normal data under genuine temporal drift on SWaT. A lightweight, CPU-only drift detector that retrains on recent data recovers skill and roughly halves false alarms.
- It demonstrates **deployability**. The full pipeline runs within Raspberry-Pi-class limits, measured by per-reading latency, model footprint, and peak memory inside a constrained container.

The rest of the paper is organised as follows. Section 2 reviews related work. Section 3 describes the two datasets. Section 4 covers the feature engineering, models, evaluation metrics, drift adaptation, and constrained runtime. Section 5 presents the results. Section 6 discusses findings and limitations, and Section 7 concludes.

2 Related Work

Anomaly detection is a mature field with a well-established taxonomy of statistical, distance-based, density-based, and classification-based approaches [2]. Classical methods remain popular because they are fast, easy to interpret, and need little tuning. Distance- and density-based detectors such as the Local Outlier Factor (LOF) score a point by the relative density of its neighbourhood [5], extending earlier density-based clustering work that separates dense regions from sparse, noise-like points [6]. Boundary methods such as the One-Class SVM learn a surface that encloses the bulk of the normal data [7]. Covariance-based methods model the normal data as one distribution and flag points with large Mahalanobis distance; the Elliptic Envelope uses the Minimum Covariance Determinant estimator so that a few contaminating points do not distort the fit [8]. Among reconstruction-based detectors, PCA reconstruction error is the linear archetype: it projects data onto a low-rank normal subspace and flags points that reconstruct poorly [9]. Isolation Forest takes a different route, isolating anomalies through random recursive partitioning on the premise that anomalies are few and different and so are easier to isolate [10].

Deep learning has also been applied widely to anomaly detection [4]. Sequence models such as LSTM encoder–decoders reconstruct normal multivariate sequences and use the reconstruction error as an anomaly score [11], extending earlier work that used plain autoencoders as nonlinear reconstruction-based detectors [12]. These models capture temporal dependencies that tabular methods miss, but they need much more data, compute, and tuning to train and deploy. Recent benchmark studies report that, on real multivariate-time-series data, deep detectors do not consistently outperform well-tuned classical methods, and

that careful feature engineering narrows or removes the gap [3, 4]. This motivates the scope adopted here. The study asks how far well-engineered features and standard detectors can go on real benchmarks, and it includes a lightweight LSTM-autoencoder as a deep baseline to measure that gap directly (Section 4.7) rather than assuming it either way.

A second line of work concerns concept drift, meaning change in the data distribution over time, and how to adapt to it through drift detection and retraining on recent data [13], for example by tracking a statistic over an adaptively resized window of recent observations [14]. Drift matters for deployed condition-monitoring systems, whose sensor baselines shift with wear and changing operating conditions. This paper links the two threads. It first evaluates classical detectors and then subjects the strongest models to drift, measuring whether lightweight adaptation recovers performance.

3 Datasets

Two public, real multivariate industrial sensor benchmarks are used.

3.1 SKAB

The Skoltech Anomaly Benchmark (SKAB) [15] was recorded from a water-circulation testbed built from a pump, pipework, and valves. Faults are induced physically, mostly by partly closing a valve to restrict flow. Each recording samples eight sensors at 1 Hz: two accelerometers (vibration RMS), motor current, system pressure, water temperature, pump-housing thermocouple temperature, supply voltage, and volumetric flow rate (RMS). The corpus has 35 CSV files. One is an anomaly-free baseline run, and 34 are fault runs in three groups: `valve1` (16), `valve2` (4), and `other` (14 miscellaneous faults). The corpus holds 46,860 rows in total. Each fault run carries a per-timestamp `anomaly` label. Each CSV is an independent experiment with its own clock, so the runs are not one continuous stream. Table 1 shows a short excerpt.

Table 1. Excerpt from a SKAB fault run (four consecutive rows). Four of the eight sensors are shown for space; each row also carries the binary `anomaly` label (0 = normal, 1 = fault).

datetime	Accel1RMS	Current	Pressure	Temp.	anomaly
2020-03-09 10:34:33	0.0271	0.871	0.055	75.50	0
2020-03-09 10:34:34	0.0270	1.301	0.055	75.54	0
2020-03-09 10:34:35	0.0268	1.285	0.383	75.66	0
2020-03-09 10:34:36	0.0268	1.365	0.055	75.58	0

3.2 SWaT

The Secure Water Treatment (SWaT) testbed [16, 17] is a full-scale six-stage water-treatment plant instrumented with 51 sensors and actuators sampled at 1 Hz. It is one of the largest public industrial control benchmarks, with about 1.44 million rows in total: a long run of normal operation followed by a run with 36 labelled attacks that physically manipulate the process. Of the 51 channels, 25 are continuous sensors (flows, tank levels, pressures, chemical analysers) and 26 are discrete actuator states (valves, pumps). Table 2 shows a short excerpt. The fully-recorded normal block used here has 395,298 rows. A further block, in which seven channels were not recorded, is dropped rather than imputed. The attack run contributes 54,621 labelled rows. Compared with SKAB (8 sensors), the much larger and strongly inter-correlated SWaT sensor set turns out to drive the cross-dataset findings in Section 5.

Table 2. Excerpt from the SWaT run (four consecutive rows). Four of the 51 channels are shown for space (25 continuous sensors are used in this study). The sampling instant is given as a relative time index t in seconds.

t (s)	FIT101	LIT101	AIT201	FIT401	label
0	2.494	817.67	262.63	1.712	Attack
1	2.536	817.95	262.63	1.712	Attack
2	2.536	817.95	262.63	1.712	Attack
3	2.570	818.46	262.63	1.712	Attack

4 Methodology

4.1 Feature Engineering

Raw sensor values discard temporal context, so each timestamp is augmented with short-horizon causal statistics. For every continuous sensor, and using only the current and past readings, the feature set adds a 5-second rolling mean, a 5-second rolling standard deviation, a 30-second rolling mean, and the first difference. The raw value is kept, and one global feature, `time_since_start`, is added. On SKAB this gives 41 features ($8 \times 5 + 1$). On SWaT the same recipe over the 25 continuous sensors gives 126 features. The discrete actuator channels are left out so that the same rolling-statistics recipe applies to both datasets and the comparison stays consistent.

Two choices protect the feature space. First, all rolling statistics are computed within a single experiment segment. A window is never allowed to cross the boundary between independent runs, or between a train and a test segment, which would create a spurious link between unrelated conditions. Second, wall-clock features such as hour-of-day and day-of-week are left out on purpose.

Including them gave a small score increase at first, but an ablation traced this to leakage. Because the SKAB test set is the chronological tail of the same files used for training, the hour within a short run is almost constant and acts as a hidden file identifier. Dropping these features lowered the headline F1 but gave a number that transfers to unseen data. All results use the leakage-free representation.

4.2 Splits and Unsupervised Training

The data is a time series, so it is never shuffled. **SKAB.** Each fault file is split chronologically into the first 80% for training and the final 20% for testing, and these portions are concatenated across files. The unsupervised training set combines the entire anomaly-free run with the rows labelled normal (that is, with no fault present) from each training portion, which keeps training free of labelled anomalies. This gives 28,368 training rows and 7,503 test rows, of which 30.0% are anomalous. **SWaT.** The normal run is split chronologically into 70% for training, 15% for validation-normal, and 15% for test-normal, and the attack run is split into validation and test halves. The model trains on normal rows only. This gives 276,708 training rows and an 86,606-row test set, of which 31.5% are anomalous. On both datasets, hyperparameters are chosen on the validation split only, and features are standardised with a scaler fit on the training set alone.

4.3 Models

Six unsupervised detectors are evaluated. All come from scikit-learn [18] except the statistical baseline, and all sit behind a common train/predict interface on the same scaled feature matrix, built with NumPy [19] and pandas [20] for array and tabular data handling. The z-score baseline flags a row $\mathbf{x}$ when any standardised feature exceeds a fixed threshold τ,

$$\max_j \left| \frac{x_j - \mu_j}{\sigma_j} \right| > \tau, \tag{1}$$

where μ_j and σ_j are the training mean and standard deviation of feature j. The One-Class SVM [7] uses an RBF kernel. Local Outlier Factor [5] runs in novelty mode so it can score held-out points. PCA reconstruction error [9] keeps the components that explain 95% of the training variance and scores by the residual error. The Elliptic Envelope [8] fits a Mahalanobis model through the Minimum Covariance Determinant. Because the features are partly collinear (rolling means track raw values) the raw covariance is singular, so the data is PCA-whitened to 10 components first, a standard way to keep the estimate numerically stable. Isolation Forest [10] scores by how shallowly a point is isolated in an ensemble of random trees.

4.4 Evaluation Metrics

Precision, recall, and F1 on the anomaly class are reported together with the confusion matrix. F1 is not used as the only headline metric. A detector that alarms on almost every row reaches recall near 1 and precision near the anomaly base rate, which produces a middling F1 even though the detector separates nothing. The Matthews correlation coefficient (MCC) [21] and balanced accuracy are therefore reported and treated as primary, since both stay informative under class imbalance where accuracy and F1 can be misleading [22]. From the confusion-matrix counts,

$$\mathrm{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}, \tag{2}$$

which is 0 for a detector whose predictions are independent of the labels, including one that flags every row, and 1 for perfect separation. Balanced accuracy is the mean of the per-class recall and equals 0.5 in the same flag-everything case. Both therefore reward only real separation of anomalies from normal operation. This distinction becomes important in the concept-drift analysis, where the static baseline turns into exactly that kind of flag-everything predictor.

4.5 Contamination Tuning

The contamination parameter, meaning the assumed anomaly fraction, is tuned on the validation split, and optima that correspond to flag-everything behaviour are rejected. On SKAB, the Isolation Forest sweep has an interior optimum at 0.35, and this shared value is applied across the contamination/ν-based models. On SWaT, selection is by MCC. There, F1 keeps rising toward the degenerate high-contamination end of the range, whereas MCC peaks at a selective value of 0.05 and then falls toward zero as the model degenerates, so MCC selection lands on an operating point that can actually discriminate.

4.6 Concept-Drift Detection and Adaptation

A streaming detector compares the feature distribution of an incoming batch against a reference distribution using the two-sample Kolmogorov–Smirnov (KS) D-statistic [23]. For feature j, with empirical CDFs F_j^{ref} and F_j^{win} over the reference set and the current window, the per-feature statistic and the batch drift score are

$$D_j = \sup_x \left| F_j^{\mathrm{ref}}(x) - F_j^{\mathrm{win}}(x) \right|, \qquad \bar{D} = \frac{1}{d} \sum_{j=1}^{d} D_j, \tag{3}$$

where d is the number of features. Intuitively, the D-statistic measures how far the current data has moved away from the training data on a scale from 0 to 1, where 0 means the two distributions are identical and 1 means they do not

overlap at all. A rising score is therefore a simple, readable drift alarm. The D-statistic is used rather than the KS p-value. With hundreds of rows per feature in each batch, the p-value drops to almost zero for even a tiny change and stops being informative, whereas the bounded D-statistic reports how large the change is and does not grow just because there are more samples [13]. When $\bar{D}$ passes a threshold, calibrated from ordinary within-regime variation and never from the test stream, the model is retrained on a sliding window of the most recent normal-labelled batches. Using a window rather than all past history lets the model forget a superseded regime that would otherwise outnumber and dilute the recent data.

Two drift scenarios are studied. On SKAB a regime shift is constructed. The initial model is trained only on anomaly-free data plus `other`-family normal data, and the test files are streamed in family order (`other` to `valve1` to `valve2`), so `valve1` and `valve2` arrive as unseen regimes. On SWaT the drift is real and temporal. A detector trained on early normal data is streamed forward through later normal operation, whose sensor baselines move over the multi-day run.

4.7 Deep Sequential Baseline

To test whether the classical detectors' low compute costs a real accuracy penalty, a lightweight LSTM [24] autoencoder is added as a deep sequential baseline, implemented in PyTorch [25]. Unlike the tabular detectors, it is given the temporal structure directly: its input is a sliding window of 30 consecutive raw (scaled) sensor readings (8 channels on SKAB, 25 on SWaT). A single-layer LSTM encoder with 64 hidden units compresses the window, a matching LSTM decoder reconstructs it, and the anomaly score for a row is the mean squared reconstruction error of the causal window ending at that row. Training uses normal windows only (Adam [26], 25 epochs), and the decision threshold is selected on the validation split by MCC while rejecting flag-everything cut-offs, the same rule used for the classical contamination. The model is kept small (about 58k parameters, 0.2 MB) so its footprint can be compared against the classical detectors, and it is evaluated per row on the identical SKAB and SWaT test splits.

4.8 Constrained-Hardware Runtime

To test edge deployability while using only commodity hardware, the pipeline runs inside a Docker container limited to one CPU core, 1 GB of RAM, and the ARM64 architecture, which is the same instruction-set family as a Raspberry Pi [27]. The measurements cover per-model training time, inference latency in milliseconds per reading, on-disk model size, the on-device cost of a drift check and a retrain, and whole-run peak memory. The effective CPU limit is read from the container control group to confirm the single-core cap. This setup reproduces a Pi's resource limits and instruction set, though not its slower per-core speed, so the latency figures are an optimistic lower bound and are reported as measured on a constrained ARM container rather than on a physical Pi.

5 Results

5.1 Model Comparison on SKAB

Table 3 and Fig. 1 give the comparison on SKAB. Isolation Forest is the strongest detector on every metric (F1 0.466, MCC 0.147), with the Elliptic Envelope next. The absolute scores are modest. The best precision is about 0.36, which means roughly two of every three alarms are false. That level is typical of unsupervised detection on real industrial data, and it is reported as measured rather than inflated. The deep LSTM autoencoder is the weakest entry here: its MCC of −0.045 is below chance and below every classical detector, an outcome examined in Section 5.3. Fig. 2 shows the confusion matrices, and Figs. 3–4 illustrate Isolation Forest's predictions against ground truth on one fault run.

Table 3. Detector comparison on the SKAB test set, sorted by MCC (contamination 0.35 for the classical models). LSTM-AE is the deep sequential baseline. Best per column in bold.

Model	Precision	Recall	F1	MCC	bAcc
Isolation Forest	**0.362**	**0.656**	**0.466**	**0.147**	**0.580**
Elliptic Envelope	0.342	0.539	0.418	0.086	0.547
z-score	0.329	0.548	0.411	0.063	0.534
One-Class SVM	0.326	0.537	0.406	0.055	0.530
LOF	0.321	0.460	0.378	0.040	0.522
PCA-recon	0.305	0.470	0.370	0.010	0.505
LSTM-AE (deep)	0.251	0.127	0.169	−0.045	0.482

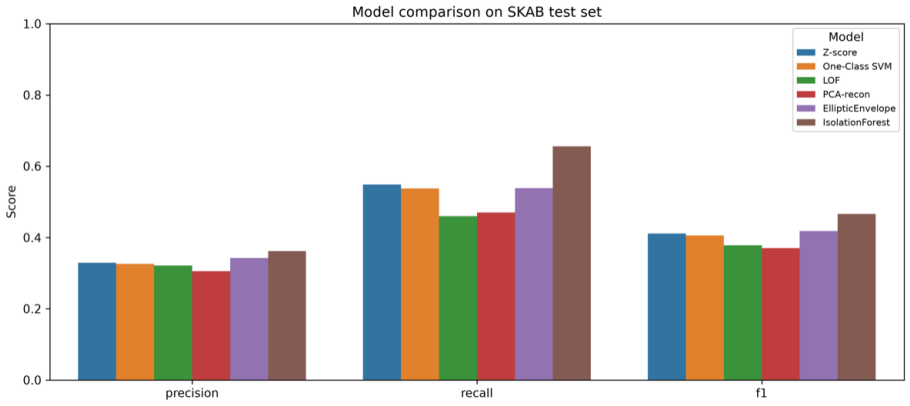


Fig. 1. Precision, recall, and F1 for the six models on the SKAB test set.

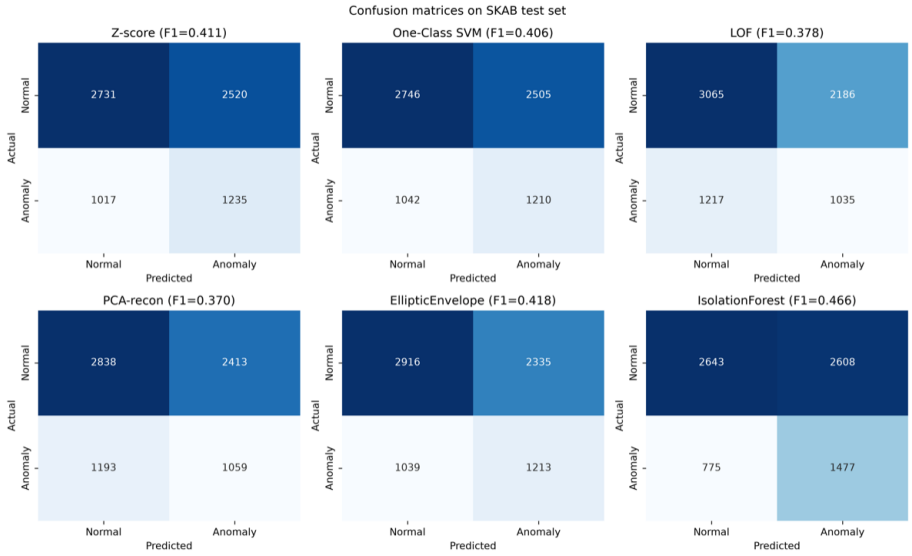


Fig. 2. Confusion matrices on the SKAB test set for all six models.

5.2 Model Comparison on SWaT

Table 4 and Fig. 5 give the comparison on SWaT (contamination 0.05 for the classical models). The ranking changes in two ways. Among the classical detectors, the correlation-based Elliptic Envelope now leads by a wide margin (MCC 0.766, F1 0.839), followed by the One-Class SVM, with Isolation Forest, the SKAB winner, in third place. The deep LSTM autoencoder does better still and tops the table (MCC 0.841, precision 0.974), the reverse of its behaviour on SKAB. Absolute performance is much higher than on SKAB across the board, which fits the richer sensor set providing stronger separation.

Table 4. Detector comparison on the SWaT test set, sorted by MCC (contamination 0.05 for the classical models). The LSTM-AE deep baseline leads. Best per column in bold.

Model	Precision	Recall	F1	MCC	bAcc
LSTM-AE (deep)	**0.974**	0.805	**0.881**	**0.841**	**0.897**
Elliptic Envelope	0.846	0.833	0.839	0.766	0.881
One-Class SVM	0.653	0.896	0.755	0.634	0.838
Isolation Forest	0.546	0.823	0.657	0.473	0.754
z-score	0.349	0.955	0.511	0.180	0.567
LOF	0.324	**0.994**	0.489	0.105	0.520
PCA-recon	0.316	0.858	0.461	0.002	0.501

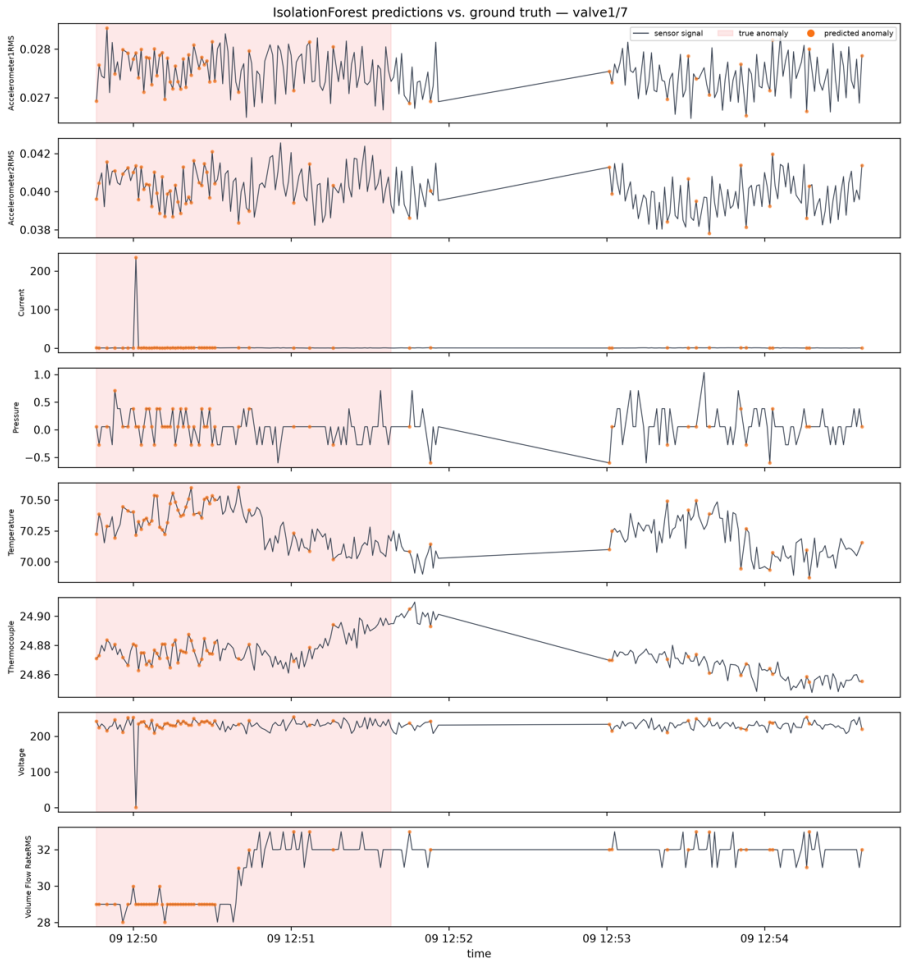


Fig. 3. Isolation Forest predictions (orange) against ground-truth anomaly regions (red shading) across the sensors of a SKAB fault run.

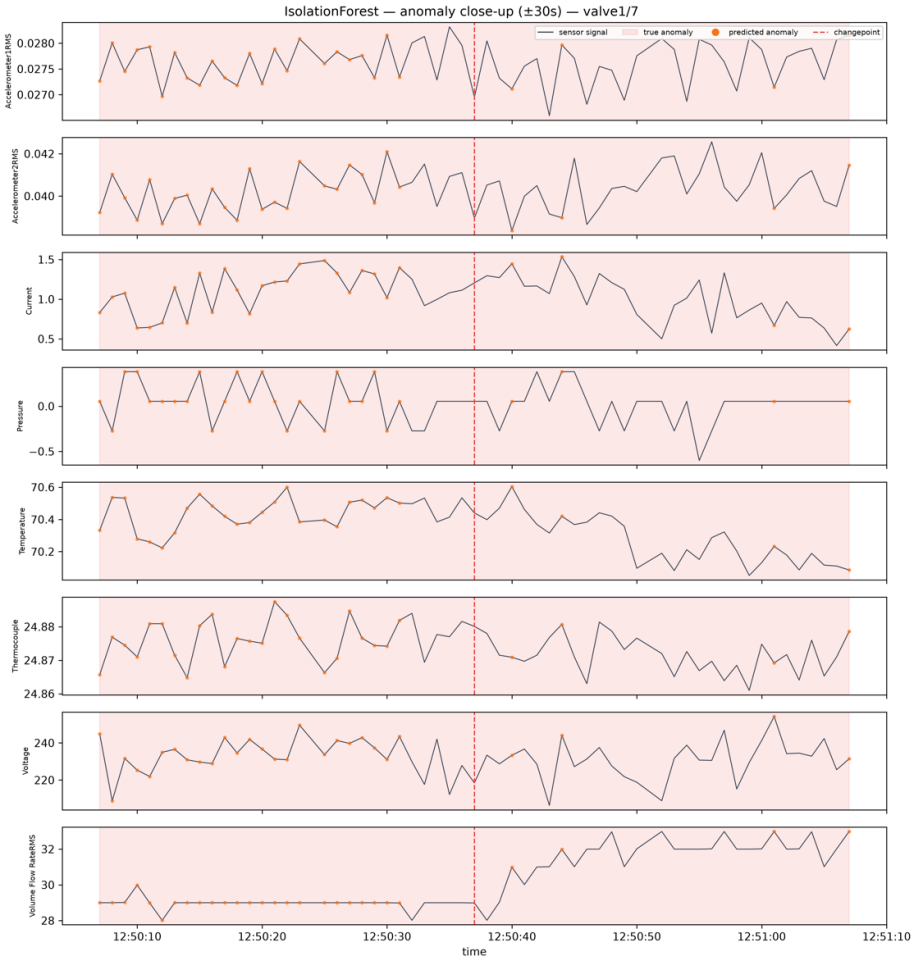


Fig. 4. Close-up around the first changepoint of the run in Fig. 3.

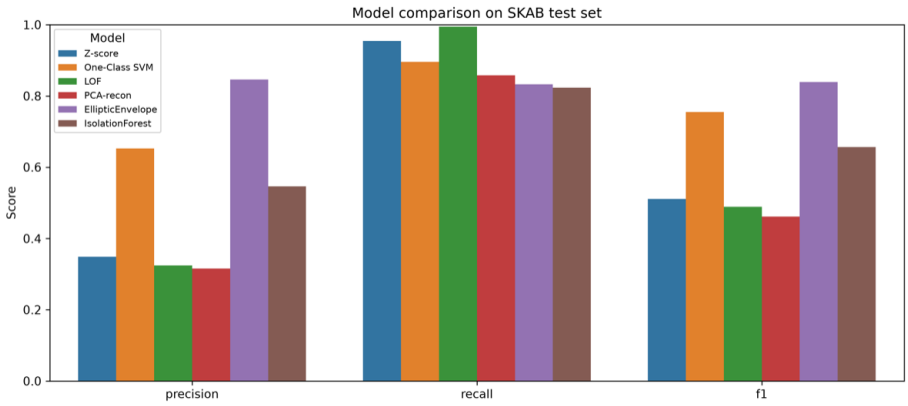


Fig. 5. Precision, recall, and F1 for the six models on the SWaT test set.

5.3 The Best Detector Is Dataset-Dependent

Fig. 6 places the two datasets side by side by MCC. The winner changes. Isolation Forest is strongest on SKAB, and the Elliptic Envelope is strongest on SWaT, there by a wide margin (0.766 against 0.473), whereas the two are close on SKAB. This is not accidental. It follows from the datasets. The 51 SWaT sensors move together in tight joint relationships, and an attack that disturbs the process breaks those relationships even when no single sensor looks extreme. That broken joint structure is exactly what a covariance-based detector like the Elliptic Envelope reads directly. SKAB's 8 sensors offer far less joint structure to use, so the general-purpose partitioning of Isolation Forest is at least as good. In practice a model called best on one benchmark should not be assumed best on another, which makes multi-dataset evaluation necessary rather than optional.

The deep LSTM autoencoder makes the same point more sharply. It is the best detector of all on SWaT (MCC 0.841), yet on SKAB it falls below chance (MCC −0.045) and below every classical detector, even after training to convergence on denser windows. The reason is again dimensionality and data volume. On SWaT the model has 25 correlated channels and hundreds of thousands of normal windows, enough to learn a faithful temporal reconstruction so that genuine attacks stand out as large errors: their median reconstruction error is about three orders of magnitude above normal, cleanly separated by the threshold (Fig. 7, right). On SKAB it has only 8 channels and an order of magnitude fewer windows, too little for the recurrent model to learn a normal manifold that generalises, so the normal and anomaly reconstruction errors overlap almost completely (Fig. 7, left). Training converged in both cases (Fig. 8), so the SKAB failure is not an artefact of under-training; the loss plateaus and the error simply does not distinguish the classes. The deep model does not remove the dataset dependence. It amplifies it, and at a cost: it needs a deep-learning runtime far larger

than scikit-learn, benefits from a GPU for training, and its per-reading inference is several times slower than the classical leaders (Section 5.5), even though its saved size (0.2 MB) is comparable. Lightweight classical detectors therefore remain the more robust and deployable choice across both datasets, while a deep model pays off only where the data is high-dimensional and abundant and the runtime budget allows it.

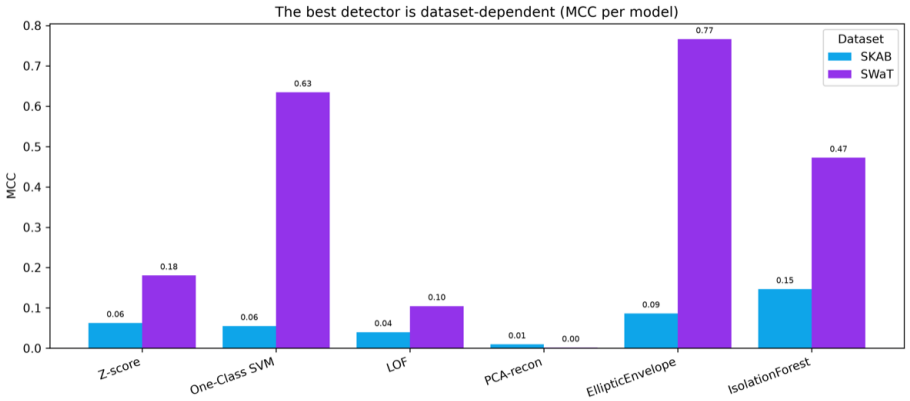


Fig. 6. Per-model MCC on SKAB versus SWaT, including the LSTM-autoencoder deep baseline. The strongest model differs by dataset (Isolation Forest on SKAB, the deep model on SWaT), and the deep model itself swings from best on SWaT to below chance on SKAB.

5.4 Concept-Drift Adaptation

The comparisons above evaluate each detector once, on a fixed distribution. A deployed detector meets data that drifts. This section subjects Isolation Forest to drift and asks whether the lightweight adaptation of Section 4.6 counteracts the resulting degradation.

SKAB (regime shift) On the unseen valve regimes the static detector has no discriminative skill. It reaches MCC 0.000 and balanced accuracy 0.500, no better than chance, and it gets there by flagging almost everything, so its F1 of 0.43 is an artefact of the flag-everything behaviour that F1 fails to penalise. Retraining on drift recovers real skill on those regimes (MCC 0.000 to 0.106, balanced accuracy 0.500 to 0.558) and cuts the false-alarm load (precision 0.285 to 0.336). Table 5 and Fig. 11 summarise this.

SWaT (genuine temporal drift) SWaT's normal operation drifts over its multi-day run. Fig. 9 quantifies this. A detector trained on early normal data sees

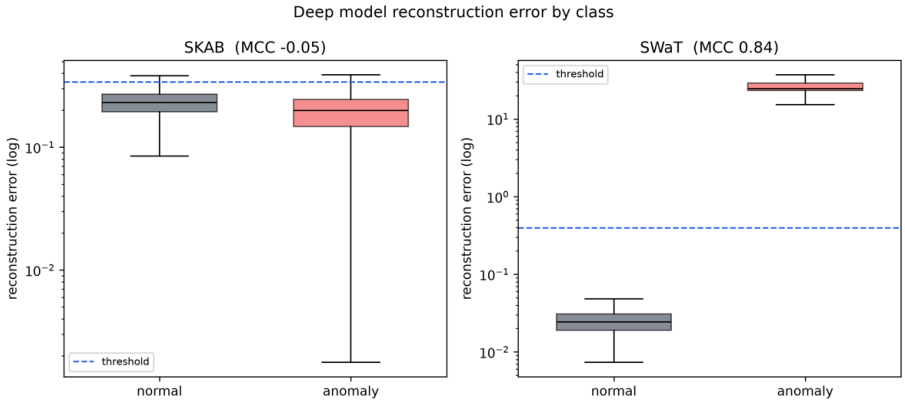


Fig. 7. Deep-model reconstruction error by class (log scale, box plots, fliers hidden). On SKAB the normal and anomaly errors overlap, so the score cannot separate the classes. On SWaT the anomaly errors sit far above the normal errors and above the decision threshold, which is why the deep model leads there.

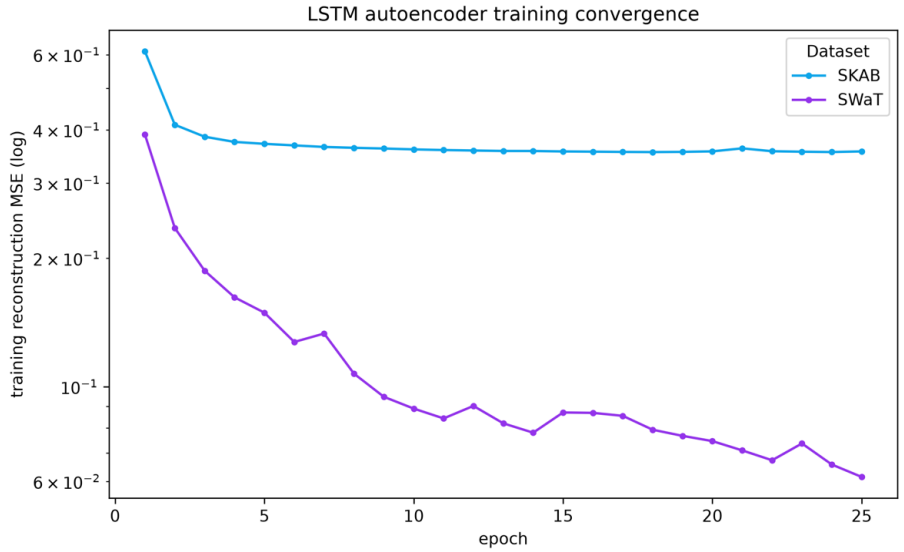


Fig. 8. LSTM-autoencoder training convergence (reconstruction MSE per epoch). Both runs converge; the SKAB loss plateaus, confirming its poor detection is not due to under-training.

Table 5. SKAB drift: static vs. adaptive by regime (mean over batches). On the unseen valve regimes the static model has zero skill (MCC 0.00); adaptation restores it.

Regime	Track	MCC	bAcc	F1
`other` (seen)	static	−0.027	0.484	0.185
	adaptive	−0.033	0.485	0.177
valve (**unseen**)	static	0.000	0.500	0.430
	adaptive	**0.106**	**0.558**	0.414
whole stream	static	−0.011	0.493	0.329
	adaptive	**0.049**	**0.528**	0.317

its false-alarm rate on later normal data climb from 8% toward 100% as the sensor baselines move, tracked by a rising KS drift score. Under the static-versus-adaptive stream (Table 6, Fig. 10), retraining on drift more than doubles skill (MCC 0.242 to 0.594; balanced accuracy 0.630 to 0.826), cuts the false-alarm rate on normal data from 73% to 24%, and keeps most attack detection (recall 0.99 to 0.90). This drift is real rather than constructed, and the gain is larger than on SKAB.

Table 6. SWaT drift: static vs. adaptive (mean over 34 streamed batches, 25 retrains). Adaptation halves false alarms and doubles skill while keeping detection.

Metric	Static	Adaptive
MCC	0.242	**0.594**
Balanced accuracy	0.630	**0.826**
Precision	0.312	**0.583**
False-alarm rate (normal)	0.734	**0.244**
Recall (attacks)	0.993	0.896

5.5 Deployability on Constrained Hardware

Table 7 and Fig. 12 report the pipeline run inside the one-core, 1 GB, ARM64 container. The whole workload finishes with peak memory at 484 MB, under half of the 1 GB cap, so it fits with room to spare. Per-reading inference stays below a millisecond for every model and sits near two microseconds for the two strongest (0.0017 ms for the Elliptic Envelope, 0.0019 ms for Isolation Forest). The leading models are also small on disk (285 KB and 1.1 MB). The cost of adaptation is likewise low: a KS drift check takes about 60 ms per batch and a full retrain about 0.11 s. A bounded SWaT inference test inside the same container reproduces the cross-dataset ordering (Elliptic Envelope MCC 0.81 against Isolation Forest 0.66), so the constrained runtime does not change the

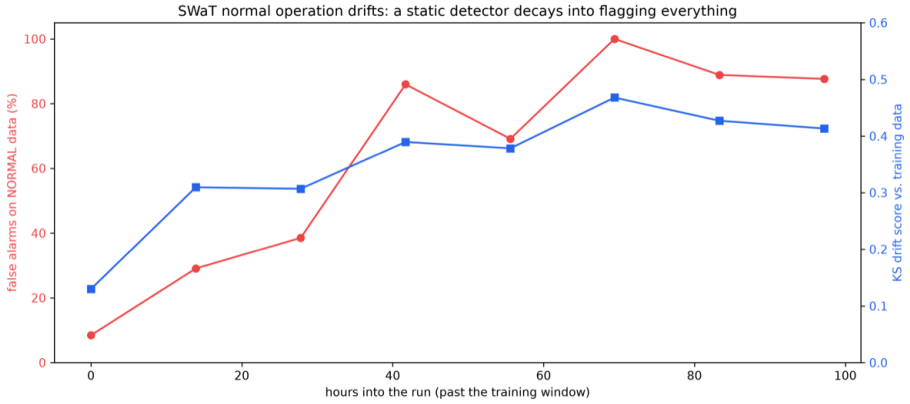


Fig. 9. SWaT normal operation drifts. The static detector's false-alarm rate on normal data (red) climbs from 8% toward 100% as the KS drift score (blue) rises, which motivates adaptation.

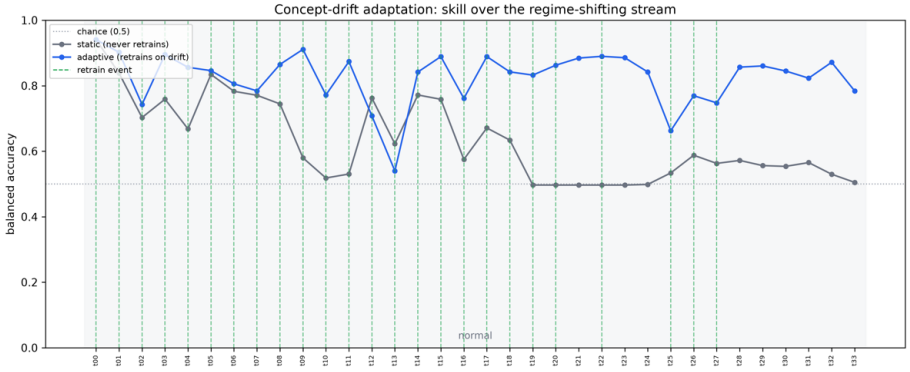


Fig. 10. SWaT concept-drift stream. Balanced accuracy over time for the static (grey) and adaptive (blue) detectors, with dashed lines marking retrains. The static detector decays toward the 0.5 chance floor as normal drifts, while the adaptive detector re-anchors after each retrain.

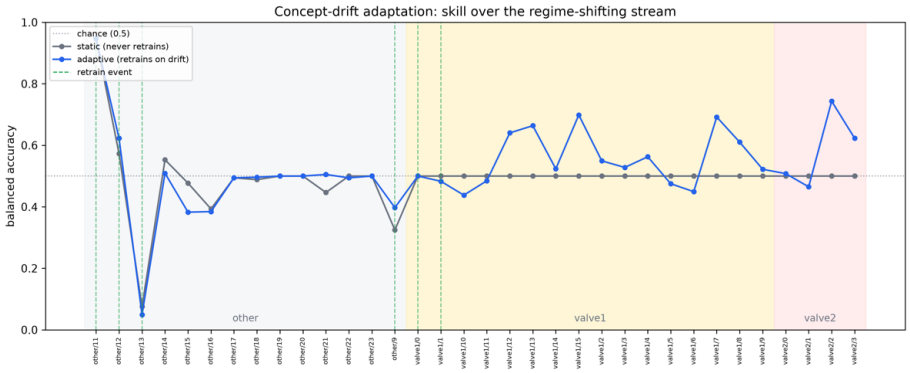


Fig. 11. SKAB concept-drift stream (balanced accuracy). Across the unseen `valve1` and `valve2` regimes the static detector (grey) sits on the 0.5 chance floor, while the adaptive detector (blue) recovers skill.

result. These figures support edge deployability, subject to the caveat in Section 4 that a core faster than a physical Pi makes the latency an optimistic bound. The deep LSTM autoencoder is not included in the container run: its deep-learning runtime alone exceeds the lightweight footprint measured here, and it benefits from a GPU for training. On the host its saved model is small (0.2 MB), but per-reading inference is several times slower than the classical leaders. For constrained deployment the classical detectors therefore remain preferable, the deep model being justified only by the accuracy it buys on a high-dimensional dataset such as SWaT.

Table 7. Deployability inside a constrained ARM container (1 CPU, 1 GB RAM): per-reading inference latency and on-disk model size. Whole-run peak memory was 484 MB of the 1024 MB cap.

Model	Inference (ms/reading)	Size (KB)
z-score	< 0.001	1
Isolation Forest	0.0019	1,134
Elliptic Envelope	0.0017	285
PCA-recon	0.0132	8
LOF	0.0901	13,963
One-Class SVM	0.2415	3,377

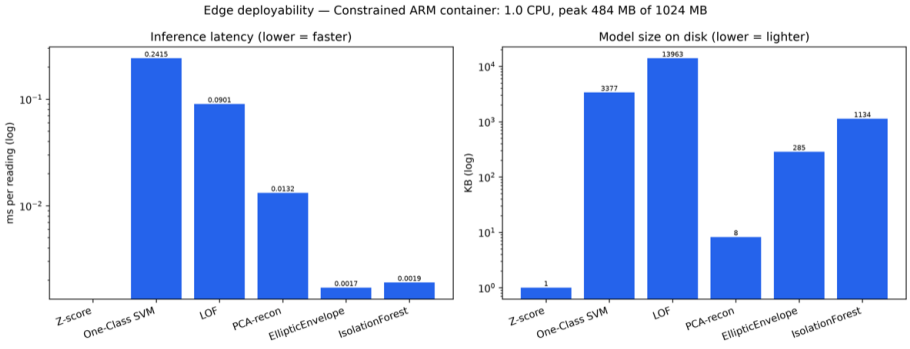


Fig. 12. Edge deployability inside a constrained ARM container (1 CPU, peak 484 MB of 1024 MB). Per-model inference latency and on-disk size are shown on a log scale. The leading detectors are both fast and compact.

6 Discussion and Limitations

Three results stand out, and each is reported as measured, including where it revises an earlier expectation.

The first is that no single model wins. Across the two datasets the best classical detector changed, and the change tracks sensor count and correlation structure rather than being incidental. The deep baseline behaves the same way, only more extremely: best of all on SWaT, below chance on SKAB. This weakens any universal best-model claim and argues for evaluation on more than one dataset.

The second is that static detectors fail quietly under drift. On both datasets a static detector degrades, to chance on the unseen SKAB regimes and toward flagging all normal data on SWaT, and F1 partly hides the failure. The degradation is only fully visible under a metric that refuses to credit flag-everything behaviour, which is itself a lesson about metric choice. Lightweight, CPU-only adaptation counteracts the failure.

The third is that adaptation restores modest, not perfect, skill. On SKAB the adaptive detector moves from no skill to some skill (MCC 0.00 to 0.11). On SWaT the gain is larger (MCC 0.24 to 0.59) and false alarms roughly halve. The value of the SKAB result is qualitative. The static model collapses to exactly chance (MCC 0.00), and a simple, label-free statistic, the KS D-statistic, is enough to detect that collapse and trigger recovery automatically. That mechanism is a structural foundation on which richer retraining strategies can build, and the small absolute recovery on SKAB reflects the difficulty of that particular regime-shift proxy rather than a limit of the mechanism. The contribution is the demonstration that a static detector fails under drift and that cheap adaptation counteracts it, not a claim of a deployment-ready detector in absolute terms.

Several limitations bound these claims. Absolute performance on SKAB is modest, with best precision near 0.36, so the contribution is a fair comparison

and an adaptation mechanism, not a state-of-the-art detector. Both datasets are water systems, so generalisation to other machine classes such as bearing wear, turbomachinery, or electrical faults is not established. The SWaT data was obtained through a public re-upload; the official iTrust source should be cited and its terms observed for any formal release. The deep baseline tested here is a single lightweight LSTM autoencoder [11]; larger or better-tuned sequence models might close the gap on SKAB or widen the lead on SWaT, and the drift adaptation was studied only for the classical detectors, not the deep one. The SKAB drift scenario is a constructed regime-shift proxy built from fault families, while the SWaT drift is genuine; checking against progressive physical wear is left for future work. Finally, the edge evaluation constrains resources on an ARM host but is not a physical Raspberry Pi, so the latency is an optimistic bound, and on-device timing on real hardware is future work.

7 Conclusion

This paper presented a controlled, reproducible, and deliberately broad study of unsupervised anomaly detection for multivariate industrial sensors. Seven detectors were evaluated: six classical models (a z-score baseline, One-Class SVM, Local Outlier Factor, PCA reconstruction, the Elliptic Envelope, and Isolation Forest) and one lightweight LSTM-autoencoder deep baseline. They were tested on two real benchmarks of very different scale, SKAB (eight sensors, about 47k rows) and SWaT (fifty-one channels, about 1.44 million rows), under a single leakage-free feature set, a strictly chronological protocol, and five metrics that include the discrimination-aware MCC and balanced accuracy. Every detector was then examined under two further dimensions that a plain comparison omits: distribution drift over time, and execution inside constrained edge hardware.

Four concrete outcomes follow. First, the best classical detector depends on the dataset. Isolation Forest leads on SKAB (MCC 0.147) while the correlation-based Elliptic Envelope leads on SWaT (MCC 0.766), so a model declared best on one benchmark does not transfer to another. Second, the deep baseline sharpens rather than removes this dependence. It tops every detector on SWaT (MCC 0.841) yet falls below chance on the smaller SKAB (MCC -0.045), even after training to convergence, which shows that a deep model repays its extra cost only where the data is high-dimensional and abundant. Third, a deployed static detector degrades under concept drift, reaching chance-level skill on unseen SKAB regimes and driving its false-alarm rate on normal SWaT data from 8% toward 100%. A lightweight, CPU-only detector that senses drift with a Kolmogorov–Smirnov statistic and retrains on recent data restores skill (SWaT MCC 0.242 to 0.594), halves false alarms (73% to 24%), and keeps most attack recall (0.99 to 0.90). Fourth, the whole pipeline, spanning the classical models and the drift adaptation, runs inside a Raspberry-Pi-class container limited to one CPU core, 1 GB of RAM, and the ARM64 architecture, with per-reading inference below a millisecond and peak memory of 484 MB; the constrained run reproduces the same cross-dataset ordering as the unconstrained host.

Taken together, these results contribute more than a single benchmark number. They provide a transparent, multi-dataset and multi-model template for careful unsupervised evaluation, and direct evidence that drift adaptation is both necessary and feasible on commodity edge hardware. The immediate next step is on-device validation on physical edge hardware such as a Raspberry Pi, to obtain true bare-metal latency under real clock, thermal, and I/O constraints rather than the optimistic bound reported here. Further work will extend the evaluation to other machine classes such as rotating machinery and electrical faults, to newer same-domain benchmarks such as WADI [28], apply the drift adaptation to the deep model as well, and test it against progressive physical wear.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Das, O., Das, D.B., Birant, D.: Machine learning for fault analysis in rotating machinery: a comprehensive review. Heliyon **9**(6), e17584 (2023)
2. Chandola, V., Banerjee, A., Kumar, V.: Anomaly detection: a survey. ACM Computing Surveys **41**(3), 1–58 (2009)
3. Belay, M.A., Blakseth, S.S., Rasheed, A., Salvo Rossi, P.: Unsupervised anomaly detection for IoT-based multivariate time series: existing solutions, performance analysis and future directions. Sensors **23**(5), 2844 (2023)
4. Chalapathy, R., Chawla, S.: Deep learning for anomaly detection: a survey. arXiv:1901.03407 (2019)
5. Breunig, M.M., Kriegel, H.P., Ng, R.T., Sander, J.: LOF: identifying density-based local outliers. In: Proceedings of the ACM SIGMOD International Conference on Management of Data, pp. 93–104 (2000)
6. Ester, M., Kriegel, H.P., Sander, J., Xu, X.: A density-based algorithm for discovering clusters in large spatial databases with noise. In: Proceedings of the 2nd International Conference on Knowledge Discovery and Data Mining (KDD), pp. 226–231 (1996)
7. Schölkopf, B., Platt, J.C., Shawe-Taylor, J., Smola, A.J., Williamson, R.C.: Estimating the support of a high-dimensional distribution. Neural Computation **13**(7), 1443–1471 (2001)
8. Rousseeuw, P.J., Van Driessen, K.: A fast algorithm for the minimum covariance determinant estimator. Technometrics **41**(3), 212–223 (1999)
9. Shyu, M.L., Chen, S.C., Sarinnapakorn, K., Chang, L.: A novel anomaly detection scheme based on principal component classifier. In: Proceedings of the IEEE ICDM Workshop on Foundations and New Directions of Data Mining, pp. 172–179 (2003)
10. Liu, F.T., Ting, K.M., Zhou, Z.H.: Isolation forest. In: Proceedings of the 8th IEEE International Conference on Data Mining (ICDM), pp. 413–422 (2008)
11. Malhotra, P., Ramakrishnan, A., Anand, G., Vig, L., Agarwal, P., Shroff, G.: LSTM-based encoder-decoder for multi-sensor anomaly detection. In: Proceedings of the ICML Anomaly Detection Workshop (2016). arXiv:1607.00148
12. Sakurada, M., Yairi, T.: Anomaly detection using autoencoders with nonlinear dimensionality reduction. In: Proceedings of the MLSDA 2014 2nd Workshop on Machine Learning for Sensory Data Analysis, pp. 4–11 (2014)

13. Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., Bouchachia, A.: A survey on concept drift adaptation. ACM Computing Surveys **46**(4), 1–37 (2014)
14. Bifet, A., Gavaldà, R.: Learning from time-changing data with adaptive windowing. In: Proceedings of the 2007 SIAM International Conference on Data Mining (SDM), pp. 443–448 (2007)
15. Katser, I.D., Kozitsin, V.O.: Skoltech Anomaly Benchmark (SKAB). Kaggle (2020). https://doi.org/10.34740/KAGGLE/DSV/1693952
16. Goh, J., Adepu, S., Junejo, K.N., Mathur, A.: A dataset to support research in the design of secure water treatment systems. In: Proceedings of the 11th International Conference on Critical Information Infrastructures Security (CRITIS), pp. 88–99 (2016)
17. Mathur, A.P., Tippenhauer, N.O.: SWaT: a water treatment testbed for research and training on ICS security. In: Proceedings of the International Workshop on Cyber-physical Systems for Smart Water Networks (CySWater), pp. 31–36 (2016)
18. Pedregosa, F., et al.: Scikit-learn: machine learning in Python. Journal of Machine Learning Research **12**, 2825–2830 (2011)
19. Harris, C.R., et al.: Array programming with NumPy. Nature **585**(7825), 357–362 (2020)
20. McKinney, W.: Data structures for statistical computing in Python. In: Proceedings of the 9th Python in Science Conference (SciPy), pp. 56–61 (2010)
21. Matthews, B.W.: Comparison of the predicted and observed secondary structure of T4 phage lysozyme. Biochimica et Biophysica Acta **405**(2), 442–451 (1975)
22. Powers, D.M.W.: Evaluation: from precision, recall and F-measure to ROC, informedness, markedness and correlation. Journal of Machine Learning Technologies **2**(1), 37–63 (2011)
23. Massey, F.J.: The Kolmogorov-Smirnov test for goodness of fit. Journal of the American Statistical Association **46**(253), 68–78 (1951)
24. Hochreiter, S., Schmidhuber, J.: Long short-term memory. Neural Computation **9**(8), 1735–1780 (1997)
25. Paszke, A., et al.: PyTorch: an imperative style, high-performance deep learning library. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 32, pp. 8024–8035 (2019)
26. Kingma, D.P., Ba, J.: Adam: a method for stochastic optimization. In: Proceedings of the 3rd International Conference on Learning Representations (ICLR) (2015)
27. Merkel, D.: Docker: lightweight Linux containers for consistent development and deployment. Linux Journal **2014**(239) (2014)
28. Ahmed, C.M., Palleti, V.R., Mathur, A.P.: WADI: a water distribution testbed for research in the design of secure cyber physical systems. In: Proceedings of the 3rd International Workshop on Cyber-Physical Systems for Smart Water Networks (CySWater), pp. 25–28 (2017)